Step 9: Telling the result: memo, dashboard, front page (ticket PPA-9)

> **Numbers on this page (update for v1.1).** This page was written with the illustrative forward of 85 EUR/MWh (v1.0). Since v1.1 the model uses the market forward curve (EEX Cal-27 base 140.21 EUR/MWh on 9 Oct 2026, Cal-28 104.75, Cal-29 87.58; see Step 8, section 4.6), and since v1.2 the market's seasonal shape and a solar cannibalisation trend (section 4.7), so prices and fair prices are higher. The methods and lessons are unchanged; current numbers are in the reports and the README, and the exercise checks use the current data.
>
> **Time needed:** about 4 hours. Concepts take 45 minutes, code tour and findings 30 minutes, hands-on, deployment and exercises 2 hours, to-dos 45 minutes. **You will be able to:** write a one-page memo that leads with the answer, build a dashboard whose logic is tested and whose numbers cannot drift from the model, turn a README into a front page, deploy the dashboard, and tell the whole project in two minutes.

Contents: 0. Scenario · 1. Concepts · 2. Architecture · 3. Code tour · 4. Findings · 5. Hands-on · 6. Results · 7. Exercises · 8. Quiz · 9. Interview · 10. Real work · 11. To-dos · 12. Glossary · 13. Sources

---

0. The scenario

Thursday morning, a message from the managing director:

> *"I have ten minutes before the investment committee. What is the price, what does it buy us, and what could go wrong? And send me something I can play with afterwards. Ticket PPA-9."*

Steps 1 to 7 produced the answer and a lot of evidence for it. None of it helps if the reader cannot find the answer in thirty seconds, or cannot check what happens when an assumption changes. Step 9 presents the work: a one-page memo, a dashboard with knobs, and a README that works as a front page for anyone opening the repository. (Step 8, hedging with market forwards, comes after: the public version had to be complete first. Cutting scope or reordering steps to hold a date is a real-project decision too.)

Ticket PPA-9: acceptance criteria (all met today)

- ☑ A one-page memo generated from the Step 7 results (Markdown and PDF).
- ☑ A dashboard: the answer, the price stack, the producer's risk by structure, and adjustable assumptions; its logic tested without Streamlit; a smoke test of the app.
- ☑ Runs from committed results only, so it can be deployed without the data pipeline.
- ☑ A README that leads with the answer, with a CI badge and the steps at a glance.
- ☑ Learning page, notebook section, exercises.

---

1. Concepts you need today

1.1 Bottom line first

Managers read the first two sentences. Put the answer there, then the reasons, then the details: the *pyramid principle* (Minto), or BLUF ("bottom line up front") in military writing. The memo's first paragraph therefore says: the price range, what it buys, what it costs. Everything else supports it.

A good bottom line is **a decision, a number and a trade-off**: "negotiate between 33.3 and 41.1 EUR/MWh; the PPA cuts our cash flow at risk by 86% for about 6.3 EUR/MWh."

1.2 Numbers that cannot drift

A memo written by hand goes stale the moment a model changes. Here the memo, the dashboard and the README all come from **one results file** (`ppa_pricing.json` → `reports/results.json`), written by the code that computed the numbers. Change an assumption, run `make all`, and every place shows the new number. In a company this is the difference between a report and a model that someone has to re-key every month.

1.3 A dashboard is a model with knobs

The dashboard does not run the Monte Carlo again (that takes data the cloud does not have). It exposes the **cheap** parts of the model, the price stack and the ten-year price, and keeps the expensive parts (scenarios, premia) as computed inputs. Choosing which knobs to expose is a modelling decision:

Knob	Why it is there
Which past represents the delivery year	The biggest lever on the solar price (Step 6)
Forward shift, capture-rate trend	The next two biggest levers (Step 7)
Tenor, discount rate, degradation	Contract and asset terms a buyer will ask about
Balancing, credit	Stated assumptions; the reader may have better numbers

The defaults are the Step 7 base case, so the dashboard opens on exactly the memo's answer.

1.4 Test the logic, smoke-test the screen

All calculations live in `ppa_lab.dashboard` and are unit-tested like any other module. The Streamlit file only lays out widgets and charts. One smoke test (`streamlit.testing.v1.AppTest`) runs the app headless, moves a slider and checks that the price reacts and nothing raises. CI installs only the dev tools, so that test is skipped there; the logic tests run everywhere.

1.5 The README is the front page

A reviewer opens the repository, reads the first screen and decides whether to read on. The README now starts with the question, the answer, one chart, a table of the seven steps with links, and how to run it. The per-step details are still there, further down.

---

2. What was built: architecture

```
Step 7: data/processed/ppa_pricing.json  (strict JSON: premia, stack, structures, tenor,
assumptions)
            |
analysis/step09_memo.py  ──►  reports/step09_memo.md + .pdf       (`ppa-lab memo`)
            └──────────────►  reports/results.json   (committed copy for the dashboard)
                                      |
ppa_lab/dashboard.py  load_results · default_assumptions · offer · structures_table  (tested)
                                      |
app/streamlit_app.py  sidebar knobs → metrics, price stack, year table, structures, steps
app/requirements.txt  streamlit, numpy, pandas, matplotlib  (Streamlit Community Cloud)
```

---

3. Code tour

3.1 `analysis/step09_memo.py`

`memo_text` builds the memo from the results: the bottom line (range, risk reduction, cost against spot sales), the price stack for both parks, what the producer keeps and gives away, the three biggest levers (read from the Step 7 sensitivity table), recommendations, method and limits. `render_pdf` turns it into an A4 PDF with headless Chrome when Chrome and the `markdown` package are available, and skips otherwise (CI has neither).

3.2 `ppa_lab/dashboard.py`

`offer(results, park, assumptions)` rebuilds the price stack with a **trend layer** (the difference between the tenor price and the first year's price) and the year-by-year table, reusing `pricing.risk.tenor_price`. The analog-pool knob switches between the configured pool and the whole window, both stored by Step 7.

3.3 `app/streamlit_app.py`

Sidebar knobs, four metrics, four tabs. It imports `ppa_lab` from `src/` when the package is not installed, so Streamlit Community Cloud needs only `app/requirements.txt`.

3.4 Tests

Test file	Protects against
`test_dashboard.py` (5)	defaults that do not reproduce the Step 7 answer; a stack that does not add up; a pool or forward knob without effect; a missing results file without guidance; wrong units in the structures table
`test_step09_memo.py` (2)	a memo without the answer, the range, the CFaR reduction or the levers; a memo longer than a page; results not published; the missing-results message
`test_app.py` (1)	an app that crashes, or does not react to its knobs (skipped without Streamlit)
`test_step07_report.py` (+3 assertions), `test_cli.py` (+1)	NaN in the JSON, missing assumptions or whole-window values; `ppa-lab memo` and `all`

---

4. Five real findings

4.1 The cost of certainty is small, the certainty is large

Against selling at spot, the PPA costs the producer the buyer's premium and the credit charge, **6.3 EUR/MWh** or **EUR 0.35 m a year** on 55 GWh (balancing is paid either way). It removes **EUR 1.24 m** of cash flow at risk (from 1.44 to 0.20). That ratio, not the price alone, is what a CFO decides on.

4.2 A fair price wins only half the time

At the fair strike the PPA earns more than merchant sales in **51%** of the scenarios (solar and wind alike, Exercise 2). "Fair" means no one wins on average; the PPA's value is the narrower spread, not a higher expectation. A memo that says "the PPA earns more" would be wrong.

4.3 NaN is not JSON

The first results file contained `NaN` for the merchant strike: Python reads it, strict JSON parsers (and some web tools) refuse it. Step 7 now writes `null`, and a test checks for it.

4.4 A fresh environment is the honest test

The app worked in the project's `.venv`, where everything is installed. In a clean virtual environment with only `app/requirements.txt` (what Streamlit Cloud does) it needed `ppa_lab` from `src/`; the app now adds that path itself, and the check was repeated in a fresh environment: no errors, the whole-window switch gives 40.9 EUR/MWh as in Step 7.

4.5 Small ones

- **Truncated metrics:** "33.3 EUR/…" did not fit four columns; units moved into the labels.

- **Arrows that lie:** Streamlit draws an up-arrow next to any metric delta, even "+0.0 vs base case"; the context lines are now captions.
- **"–0.0"** on a flat trend bar: rounded before formatting.
- **A deprecation** (`use_container_width`) caught by running the smoke test with warnings as errors.

---

5. Hands-on: run it yourself

```
cd ".../Encavis quantitative/ppa-lab"
make test                  # all offline tests (the app smoke test runs if Streamlit is
installed)
make memo                  # reports/step09_memo.md + .pdf + reports/results.json
make app                   # the dashboard at http://localhost:8501
make notebook              # Step 9 section: 9.1 to 9.3, then your solutions
```

Deploy the dashboard (you, 10 minutes, free):

1. Go to share.streamlit.io and sign in with your GitHub account.
2. *Create app* → *Deploy a public app from GitHub*: repository `ankitpatel1661/ppa-lab`, branch `main`, main file `app/streamlit_app.py`.
3. *Advanced settings*: Python 3.11. Deploy. The first start takes a few minutes.
4. Copy the URL (for example `ppa-lab.streamlit.app`) into the README, your CV and the cover letter.

Done on 15 Oct 2026: live at ppa-lab.streamlit.app (Python 3.11). Two things the first deployment taught: a Streamlit account opened with an e-mail address must first link GitHub (*Settings* → *Linked accounts* → *Sign in with GitHub*) before it can see your repositories, and the form guesses the main file path (`streamlit_app.py`): set `app/streamlit_app.py` yourself. Every push to `main` redeploys the app.

---

6. Results: what a reader sees

- **The README** opens with the question, the answer, the price-stack chart, a table of the seven steps and how to run everything.
- **The memo** (`reports/step09_memo.md`, PDF next to it) fits one page: range 33.3-41.1 EUR/MWh, CFaR –86% for 6.3 EUR/MWh, three levers, three recommendations, method and limits.
- **The dashboard** opens on the same numbers and lets the reader move the six most important assumptions; switching to "whole 2024-2026 window" raises the solar offer to 40.9 EUR/MWh.

---

7. Exercises

Warm-up (in `exercises/step09_memo.py`, checked by `.venv/bin/pytest exercises -q`)

1. **The dashboard's first number, by hand,** from `reports/results.json`.
2. **How often does the PPA beat merchant sales?** From the Step 6 scenario file.
3. **Written:** the three-sentence summary for a managing director, and its first sentence in German.

Core

4. **The buyer's memo.** Write the bottom line from the buyer's side of the table. Which numbers change, which stay?
5. **A new knob.** Add the negative-price clause to the dashboard (logic first, with a test).
6. **Deploy** the dashboard (section 5) and put the link in the README.

▼ **Hints and solution sketches**

1. `fair_price − risk_premium − balancing − credit` from `stack_eur_mwh`: 33.31 for solar with the base costs, 66.43 for wind with 3.0 and 2.0.
2. Linked rows, `K = rev.sum() / mwh.sum()`, `(K * mwh > rev).mean()`: 0.514 (solar), 0.512 (wind).
3. Lead with the price and the decision, then what it buys (a much more predictable income), then the one assumption that could move it (how much cheaper solar hours get). German, e.g.: "*Wir empfehlen einen Festpreis zwischen 33 und 41 EUR/MWh für zehn Jahre.*"
4. The buyer's bottom line: "Pay at most 41.1 (fair) minus the premium we need for the risk we take, 5.3, minus our costs."
5. The clause's revenue share is in `results.json` (`pay_as_produced_clause`).

---

8. Self-check quiz

1. What goes into the first two sentences of a management memo?
2. Why are the memo, README and dashboard generated from one results file?
3. Why does the dashboard not rerun the Monte Carlo?
4. Which assumptions does the dashboard expose, and why those?
5. How is the dashboard tested, and why is the app test skipped in CI?
6. What does the PPA cost the producer against spot sales, and what does it buy?
7. Why does the PPA beat merchant sales in only about half the scenarios?
8. Why must a results file be strict JSON?
9. Why test a deployment in a fresh environment?
10. Why was Step 9 done before Step 8?

▼ **Answers**

1. The answer (a decision and a number) and the main trade-off.
2. So they cannot drift apart: one model run updates all three.
3. The cloud has no market or weather data, and the expensive results do not depend on the knobs it exposes; it recombines stored results instead.
4. The analog pool, forward, capture-rate trend, tenor, discount rate, degradation and costs: the biggest levers and the terms a buyer asks about.

5. The logic in `ppa_lab.dashboard` by unit tests everywhere; the app by a headless smoke test that needs Streamlit, which CI does not install.
6. About 6.3 EUR/MWh (EUR 0.35 m a year); it removes EUR 1.24 m of cash flow at risk.
7. At the fair strike the expectation is equal; the PPA narrows the spread, it does not raise the average.
8. Other tools (browsers, web parsers) reject `NaN`; strict JSON is portable.
9. Your own environment hides missing dependencies; the cloud starts from nothing.
10. The application date: the public version had to be complete and polished first; hedging adds depth that interviews can use.

---

9. Interview corner

The two-minute project story (practise it out loud):

> *"I wanted to answer the question a PPA desk answers every week: what fixed price for ten years of a solar park's output, and how much risk is left? I built it from public data. First the market data, reconciled with the official figures; then capture prices, which match the TSOs' market values; then the PPA structures as tested cash-flow functions. For volumes I modelled a 50 MWp solar and a 30 MW wind park at real sites over 31 ERA5 weather years. For prices I built an hourly model with seasonality and daily mean reversion, validated against history, and then linked it to the weather, because windy and sunny days are cheap days: ignoring that would overprice wind by 12 EUR/MWh. On 496 joint scenarios the solar PPA's fair price is 41 EUR/MWh and the indicative offer 33 after the buyer's hedged risk premium and costs; the PPA cuts our cash flow at risk by 86%. The biggest open question is how much cheaper solar hours will get, and I show that as a sensitivity rather than hiding it. Everything is tested, reproducible with one command, and there is a one-page memo and a dashboard."*

Likely questions:

- "*How would a non-quant use your work?*" The memo's first paragraph, and the dashboard to test their own assumptions; both show the same numbers as the model.
- "*What would you do next?*" Market forwards and the hedge (Step 8), then a fundamental view of solar capacity and storage for the capture-rate trend.
- "*What are you least sure about?*" The future of solar cannibalisation, and the cost assumptions; both are stated and visible in the dashboard.

---

10. How this maps to real work at Encavis

In this project	In a company
One-page memo from the model	Investment-committee and pricing memos
Dashboard with stated defaults	Pricing tools for origination, scenario dashboards for management
One results file feeding all outputs	A single source of truth (database, model store) instead of copied numbers
Logic tested, UI smoke-tested	Model validation and release testing
README as front page	Model documentation and onboarding for colleagues

---

11. Your to-dos for Step 9

- ☐ Read the memo (`reports/step09_memo.md`) as if you were the MD: what would you ask?
- ☐ Run `make app`, move every knob once, and find the combination that gives 30 EUR/MWh.
- ☐ Exercises 1-3 in `exercises/step09_memo.py`; check them in "My solutions: Step 9".
- ☐ **Deploy** the dashboard (section 5) and put the link in the README and your CV.
- ☐ **Commit Step 9** (commands below), then switch `projectready` on in your CV.
- ☐ Practise the two-minute story (section 9) twice in English and once in German.
- ☐ **German vocabulary of this step:** die Entscheidungsvorlage, die Kernaussage, die Handlungsempfehlung, die Sensitivität, die Annahme, die Spanne, das Dashboard, die Reproduzierbarkeit, die Bereitstellung (deployment), der Investitionsausschuss.

Commit (check `git status` first: no `.env`, no `M .env.example`):

```
cd ".../Encavis quantitative/ppa-lab"
git status --short
git add src tests exercises config reports docs notebooks app METHODOLOGY.md README.md
CHANGELOG.md Makefile pyproject.toml .github
git status --short
git commit -m "feat(app): one-page memo, dashboard and README front page (PPA-9)"
git push
gh run watch
```

---

12. Glossary

Term	Meaning
BLUF	Bottom line up front: the answer before the reasons
Pyramid principle	Answer, then supporting arguments, then data (Minto)
Single source of truth	One place where a number is computed; everything else reads it
Smoke test	A quick test that the whole thing runs at all
Streamlit	Python framework for data apps; Community Cloud hosts them from GitHub
Deployment	Making the app run somewhere others can reach it
Front page	The first screen a reviewer sees (here the README)

13. Sources

- Minto, B. (2009): *The Pyramid Principle: Logic in Writing and Thinking.* Pearson.
- Tufte, E. R. (2001): *The Visual Display of Quantitative Information*, 2nd ed. Graphics Press.
- Streamlit documentation: app testing (`streamlit.testing.v1.AppTest`) and Community Cloud deployment.
- Encavis preparation guide: "communication to management" in the job posting; the 60-second and two-minute project stories.